

ELEC473 Assignment 4 - Synthesising the NIOS II Processor

Ryan McKee - 201954749

August 4, 2026

Abstract

This report documents the design, synthesis and testing of a Nios II soft-core system on the Altera/Terasic DE2 (Cyclone II) development board using Quartus Prime and Qsys (Platform Designer). Part A builds a base Nios II system with on-chip memory, a JTAG UART, an interval timer, parallel I/O and a system ID peripheral, and extends it with the board's 512 KB SRAM and 8 MB SDRAM, verified using the Nios II IDE memory test utility. Part B extends this system with a custom combinational instruction, `count_leading_ones`, that counts the number of leading '1' bits in a 32-bit word in a single cycle. Its design is presented as a Mealy finite state machine, implemented in Verilog, and integrated into the system as a Nios II custom instruction slave. Its performance is measured against an equivalent C software routine, with the hardware instruction found to be approximately $8\times$ faster on average.

Chapter 1

Introduction

This assignment builds familiarity with FPGA (Field Programmable Gate Array) engineering through the use of **Qsys** [1] and the **Nios II** [3] processor. Nios II is **Altera's** [2] own 32-bit embedded processor micro-architecture, designed specifically for synthesis on FPGA. Nios II is the successor to Altera's first configurable 16-bit embedded processor, Nios, introduced in 2000. Throughout this assignment we target the Cyclone II FPGA on the Altera DE2 board, which we synthesise designs for, bench test and program over the course of the work.

Nios II offers the following design parameters:

- 32 general-purpose 32-bit registers.
- A full 32-bit instruction set, data path, and address space.
- A single-instruction 32×32 multiply and divide, producing a 32-bit result.

Nios II's soft-core nature (a soft microprocessor is a microprocessor core wholly implemented using logic synthesis, and can be targeted at different semiconductor devices containing programmable logic, e.g. FPGA or CPLD, spanning both high-end and commodity applications [4]) enables hardware engineers to specify and generate a custom Nios II core, tailored to their own application requirements.

Qsys (now known as Platform Designer) is a system integration tool for Intel Quartus Prime. It connects multiple intellectual property (IP) blocks, processors (such as Nios II), and memory interfaces using standard interconnects such as Avalon, and can be used to generate the resulting system as synthesisable HDL.

This assignment is split into three parts. **Part A** synthesises a Nios II processor system with 20 KB of on-chip memory, a timer, a JTAG UART, 8 parallel I/O pins and a system ID component, then interfaces the DE2 board's 512 KB SRAM and 8 MB SDRAM to this design. The design is tested using the memory test programs within the Nios II IDE, printing name and student ID to the terminal each time a

memory test is run. **Part B** continues from Part A, adding a custom instruction that counts the number of leading ‘1’ bits in a 32-bit word, together with a C test program that exercises the instruction. A software routine implementing the same functionality is also written and benchmarked in C, so that the custom instruction’s execution speed can be compared against an equivalent software implementation. **Part C** uses PWM to control the intensity of LEDR10.

Chapter 2

Part A - SRAM & SDRAM

2.1 System Configuration

The base system is built in Qsys/Platform Designer around the Nios II processor, with 20 KB of on-chip memory, a JTAG UART for the console, an interval timer, an 8-bit parallel I/O peripheral (`led_pio`), and a System ID peripheral. The board's SRAM and SDRAM controllers are added and wired onto the same Avalon Memory-Mapped interconnect as separate memory-mapped slaves, addressable from the Nios II data master. Figure 2.1 shows the resulting system contents, including the base addresses assigned to each component by the Qsys address map (the custom instruction slave added in Part B, `count_leading_ones`, is also visible at the bottom of the table).

To exercise both memories, the Nios II IDE memory test example program was flashed and run twice: once with the test program itself (`.text`, `.rodata`, `.rdata`, heap and stack) residing in SRAM while SDRAM was tested, and once with the program residing in SDRAM while SRAM was tested. This is necessary because the memory test is destructive to the region under test, so the code and data running the test can never overlap the region being tested, otherwise the test would corrupt itself and hang or fail.

2.1.1 Memory Testing Locations

Table 2.1 lists the address regions tested for each memory device, taken from the Qsys-assigned base addresses shown in Figure 2.1.

2.1.2 SRAM Test

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		onchip_mem	On-Chip Memory (RAM or ROM)		clocks_sys...	0x0000_0000	0x0000_4fff		
☒		cpu	Nios II Processor		clocks_sys...				
☒		reset_n	Reset Input	Double-click to export	clocks_sys...				
☒		data_master	Avalon Memory Mapped Master	Double-click to export	clocks_sys...				
☒		instruction_master	Avalon Memory Mapped Master	Double-click to export	clocks_sys...				
☒		jtag_debug_module_reset	Reset Output	Double-click to export	clocks_sys...				
☒		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		custom_instruction_master	Custom Instruction Master	Double-click to export	clocks_sys...				
☒		jtag_uart	JTAG UART		clocks_sys...	0x0000_8000	0x0000_8fff		
☒		ckl	Clock Input	Double-click to export	clocks_sys...				
☒		reset	Reset Input	Double-click to export	clocks_sys...				
☒		avlagon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		sys_clk_timer	Interval Timer		clocks_sys...	0x0000_9030	0x0000_903f		
☒		ckl	Clock Input	Double-click to export	clocks_sys...				
☒		reset	Reset Input	Double-click to export	clocks_sys...				
☒		sysid	System ID Peripheral		clocks_sys...	0x0000_9000	0x0000_901f		
☒		ckl	Clock Input	Double-click to export	clocks_sys...				
☒		reset	Reset Input	Double-click to export	clocks_sys...				
☒		avlagon_slave	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		led_pio	PIO (Parallel I/O)		clocks_sys...	0x0000_9030	0x0000_9037		
☒		ckl	Clock Input	Double-click to export	clocks_sys...				
☒		reset	Reset Input	Double-click to export	clocks_sys...				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		external_connection	Conduit	led_pio_external	clocks_sys...	0x0000_9020	0x0000_902f		
☒		sram	SRAM/SSRAM Controller		clocks_sys...				
☒		clock_reset	Clock Input	Double-click to export	clocks_sys...				
☒		clock_reset_reset	Reset Input	Double-click to export	clocks_sys...				
☒		external_interface	Conduit	sram	clocks_sys...				
☒		avlagon_slave	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		clocks	Clock Signals for DE-series Board Per...		clocks_sys...	0x0000_0000	0x0007_ffff		
☒		clk_in_primary	Clock Input	clk	clocks_sys...				
☒		clk_in_primary_reset	Reset Input	reset	clocks_sys...				
☒		sys_clk	Clock Output	Double-click to export	clocks_sys...				
☒		sys_clk_reset	Reset Output	Double-click to export	clocks_sys...				
☒		sdram_clk	Clock Output	sdram_clk	clocks_sys...				
☒		sdram	SDRAM Controller		clocks_sys...				
☒		ckl	Clock Input	Double-click to export	clocks_sys...				
☒		reset	Reset Input	Double-click to export	clocks_sys...				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	clocks_sys...				
☒		wire	Conduit	sdram	clocks_sys...	0x0100_0000	0x017f_ffff		
☒		count_leading_ones_0	Custom Instruction Slave	Double-click to export	Opcode 0	Opcode 0			count_leading_ones_0
☒		nios_custom_instruction_slave_0	Custom Instruction Slave	Double-click to export	Opcode 0	Opcode 0			count_leading_ones_0

Figure 2.1: Qsys/Platform Designer system contents, showing the Nios II processor, on-chip memory, JTAG UART, interval timer, System ID, parallel I/O, SRAM and SDRAM controllers, and the `count_leading_ones` custom instruction slave added in Part B, together with their assigned Avalon base/end addresses.

Table 2.1: Memory testing address regions

Region	Origin	Length	End Address
SRAM	0x00000000	0x80000 (512 KB)	0x0007FFFF
SDRAM	0x01000000	0x800000 (8 MB)	0x017FFFFF

<----> Nios II Memory Test. <---->

Name: Ryan McKee ID: 201954749

This software example tests the memory in your system to assure it is working properly. This test is destructive to the contents of the memory it tests. Assure the memory being tested does not contain the executable or data sections of this code or the exception address of the system.

Memory Test Main Menu

- a: Test RAM
- b: Test Flash

```
q:  Exit
-----

Select Choice (a-b): [Followed by <enter>]
-ERROR:
is an invalid entry.  Please try again

Press enter to continue...
```

```
-----
Memory Test Main Menu
-----
a:  Test RAM
b:  Test Flash
q:  Exit
-----
```

```
Select Choice (a-b): [Followed by <enter>]
A
Base address to start memory test: (i.e. 0x800000)
>
0x00000000
End Address:
>
0x007FFFFF
```

2.1.3 SDRAM Test

To test SDRAM, the BSP linker section settings (`linker.x/settings.bsp`) were edited to relocate the `.text`, `.rodata`, `.rwdata`, heap and stack sections from SDRAM to SRAM, keeping SDRAM entirely free for destructive testing. Without this change the running program's own code/data was overwritten mid-test, causing the test to hang or fail.

```
<---->  Nios II Memory Test.  <---->
Name: Ryan McKee
Student ID: 201954749
This software example tests the memory in your system to assure it
is working properly.  This test is destructive to the contents of
```

the memory it tests. Assure the memory being tested does not contain the executable or data sections of this code or the exception address of the system.

```
-----  
Memory Test Main Menu  
-----  
a:  Test RAM  
b:  Test Flash  
q:  Exit  
-----
```

Select Choice (a-b): [Followed by <enter>]

a

Base address to start memory test: (i.e. 0x800000)

>

0x1000000

End Address:

>

0x17FFFFFF

Testing RAM from 0x1000000 to 0x17FFFFFF

-Data bus test passed

-Address bus test passed

-Byte and half-word access test passed

-Testing each bit in memory device. . . passed

Memory at 0x1000000 Okay

Press enter to continue...

Chapter 3

Part B - Custom Instruction

3.1 Algorithm Design

The goal of the custom instruction is to count the number of leading ‘1’ bits in a 32-bit word, i.e. the length of the run of ‘1’s starting at the most significant bit (MSB) and ending at the first ‘0’ bit (or at the LSB, if the word is all ones). Before implementing this in Verilog, the algorithm was designed as a Mealy finite state machine (Harris & Harris style), treating the 32-bit input word as a serial bit-stream presented MSB-first.

The machine has two states: S_1 , the *counting* state, and S_2 , the *done* state. Starting in S_1 , every ‘1’ bit received increments a running counter and keeps the machine in S_1 ; the first ‘0’ bit received transitions the machine to S_2 without incrementing the counter, and the machine then remains in S_2 (absorbing any remaining bits with no further effect) since the leading-ones run is over. Figure 3.1 shows the resulting state diagram.

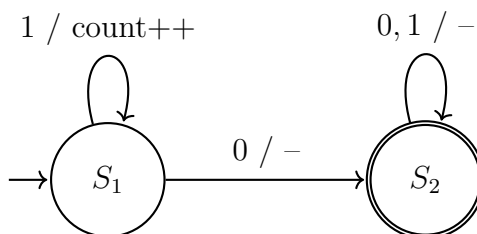


Figure 3.1: Mealy machine for the leading-ones counting algorithm. The input word is streamed MSB $\rightarrow$ LSB. S_1 (initial state) counts consecutive leading ‘1’ bits; the first ‘0’ bit moves the machine to the terminal state S_2 , after which the count no longer changes.

Formally, the machine is the 6-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0)$, where:

- $Q = \{S_1, S_2\}$ – the finite set of states.
- $\Sigma = \{0, 1\}$ – the finite input alphabet (one bit per clock/step).
- $\Gamma = \mathbb{Z}^+$ – the output alphabet (the running leading-ones count).
- $q_0 = S_1$ – the initial state.
- $\delta : Q \times \Sigma \rightarrow Q$ – the state transition function, given in Table 3.1.
- $\omega : Q \times \Sigma \rightarrow \Gamma$ – the Mealy output function: emits `count++` on $\delta(S_1, 1)$, and no output (count held) otherwise.

Table 3.1: State transition function δ

Current state	Input	Next state	Output ω
S_1	1	S_1	<code>count++</code>
S_1	0	S_2	–
S_2	0	S_2	–
S_2	1	S_2	–

Because the FSM only ever needs to hold a running count and knowledge of whether a ‘0’ bit has already been seen, it maps directly onto a combinational priority-encoder style circuit rather than a literal bit-serial shift register: the same result can be produced in a single clock cycle by evaluating the run of leading ones over the whole 32-bit word at once, halving the search space at each stage (16/8/4/2/1 bits). This is the structure used for the Verilog implementation in Section 3.3.

3.2 Software Implementation

Before implementing the custom hardware instruction, a reference software routine implementing the same leading-ones-count algorithm was written in C for the Nios II IDE, so that its speed could later be compared against the hardware instruction. `count_leading_ones()` implements the bit-serial algorithm from Figure 3.1 directly: it walks the word from bit 31 down to bit 0, incrementing a counter for each ‘1’ bit and breaking on the first ‘0’. A `PERFORMANCE_TEST` mode is included, which repeats the count over 100,000 pseudo-varied input words and reports the average execution time per call using the standard C `clock()` API.

```

    /*
    * Count Leading Ones
    *
    * This example prints 'Hello from Nios II' to the STDOUT stream, then
    * counts the number of leading '1' bits (from the MSB) in a fixed test
    * word and prints the result. It runs on the Nios II 'standard',
    * 'full_featured', 'fast', and 'low_cost' example designs. It runs with
    * or without the MicroC/OS-II RTOS and requires a STDOUT device in your
    * system's hardware.
    *
    * Define PERFORMANCE_TEST below to instead run the counting algorithm
    * repeatedly and report its average execution time (measured with the
    * standard C clock() API) rather than a single result.
    */
#include <stdio.h>
#include <stdint.h>
/* #define PERFORMANCE_TEST */
#define PERFORMANCE_TEST
#ifndef PERFORMANCE_TEST
#include <time.h>
#define PERFORMANCE_TEST_ITERATIONS 100000u
#endif
static const uint32_t word = 0xFA000000;
int count_leading_ones(uint32_t value)
{
    uint32_t leading_ones = 0;
    int i;
    for (i = 31; i >= 0; i--)
    {
        uint32_t bit = (value >> i) & 1;
        if (!bit) {
            break;
        }
        ++leading_ones;
    }
    return leading_ones;
}
#ifdef PERFORMANCE_TEST
static void run_performance_test(void)
{

```

```

clock_t start, end;
clock_t elapsed_ticks;
uint32_t elapsed_us;      /* total time in microseconds, integer */
uint32_t avg_x100_us;     /* average per call, in hundredths of a microsecond */
uint32_t i;
volatile int result = 0; /* volatile so the compiler can't optimize the loop away

printf("Running performance test (%u iterations)...\n",
      PERFORMANCE_TEST_ITERATIONS);

start = clock();
for (i = 0; i < PERFORMANCE_TEST_ITERATIONS; i++)
{
    /* Vary the input word each iteration so the result can't just
     * be computed once and cached by the compiler. */
    uint32_t test_word = word ^ i;
    result = count_leading_ones(test_word);
}
end = clock();

elapsed_ticks = end - start;
/* Convert clock ticks to microseconds using integer math only
 * (avoids the small-C-library printf %f limitation). */
elapsed_us = (uint32_t)((((uint64_t) elapsed_ticks * 1000000u) / CLOCKS_PER_SEC));
/* Average per call, scaled by 100 so we can print two decimal places
 * without any floating point. */
avg_x100_us = (uint32_t)((((uint64_t) elapsed_us * 100u) / PERFORMANCE_TEST_ITERATIONS));

printf("Performance test complete.\n");
printf("  Total time    : %lu us (%lu ticks)\n",
      (unsigned long) elapsed_us, (unsigned long) elapsed_ticks);
printf("  Avg per call  : %lu.%02lu us\n",
      (unsigned long) (avg_x100_us / 100u),
      (unsigned long) (avg_x100_us % 100u));
printf("  Last result   : %d\n", result);

if (elapsed_ticks < 10)
{
    printf("  Warning: elapsed ticks very low (%lu) - increase\n"
          "    PERFORMANCE_TEST_ITERATIONS for a more reliable\n");
}

```

```

        "    measurement; clock() resolution is limited by the\n"
        "    system timer period.\n", (unsigned long) elapsed_ticks);
    }
}
#endif
int main()
{
    printf("Hello from Nios II!\n");
#ifdef PERFORMANCE_TEST
    run_performance_test();
#else
    {
        uint32_t leading_ones = count_leading_ones(word);
        printf("leading ones %lu\n", (unsigned long) leading_ones);
    }
#endif
    return 0;
}

```

3.2.1 Software Performance Result

Running the software routine for 100,000 iterations on the Nios II system gives an average execution time of 24.63 μ s per call:

Running performance test (100000 iterations)...

Performance test complete.

Total time : 2463000 us (2463 ticks)

Avg per call : 24.63 us

Last result : 5

Because the loop breaks as soon as the first ‘0’ bit is encountered, this cost is data-dependent: words with fewer leading ones return faster, while a word of all ‘1’ bits takes the full 32 iterations.

3.3 Hardware Custom Instruction

The FSM from Figure 3.1 was re-expressed as a combinational priority-encoder tree (rather than a literal bit-serial state machine) so that it could be implemented as a Nios II *combinational* custom instruction – one that produces its result within the same clock cycle it is issued, with no internal state or extra latency. The

word is repeatedly halved: first checking whether the upper 16 bits are all '1', then the upper 8 of whichever half was selected, and so on down to a single bit, accumulating the count as each stage resolves. This directly mirrors $\delta(S_1, 1)$ (stay in S_1 , increment count) versus $\delta(S_1, 0)$ (move to S_2 , stop counting) from the FSM, but evaluates all bit positions in parallel instead of serially.

```

    module count_leading_ones (
        // Nios II custom instruction slave interface
        // "ncs_" prefix lets Platform Designer auto-recognize these as the
        // Nios II custom instruction slave interface (clk, clk_en, reset,
        // dataa, result) instead of guessing a generic Avalon-MM slave.
        ncs_clk,
        ncs_clk_en,
        ncs_reset,
        ncs_dataa,
        ncs_result
    );

input      ncs_clk;
input      ncs_clk_en;
input      ncs_reset;
input [31:0] ncs_dataa;
output [31:0] ncs_result;

// Priority tree, halving each stage | pure combinational.
// Counts leading '1' bits from the MSB of ncs_dataa.

wire msb16_all1 = (ncs_dataa[31:16] == 16'hFFFF);
wire [15:0] next16 = msb16_all1 ? ncs_dataa[15:0] : ncs_dataa[31:16];

wire msb8_all1 = (next16[15:8] == 8'hFF);
wire [7:0] next8 = msb8_all1 ? next16[7:0] : next16[15:8];

wire msb4_all1 = (next8[7:4] == 4'hF);
wire [3:0] next4 = msb4_all1 ? next8[3:0] : next8[7:4];

wire msb2_all1 = (next4[3:2] == 2'b11);
wire [1:0] next2 = msb2_all1 ? next4[1:0] : next4[3:2];

wire msb1_all1 = next2[1];
wire bit0 = msb1_all1 ? next2[0] : 1'b0;

```

```

assign ncs_result = (msb16_all1 ? 16 : 0)
                    + (msb8_all1  ? 8  : 0)
                    + (msb4_all1  ? 4  : 0)
                    + (msb2_all1  ? 2  : 0)
                    + (msb1_all1  ? 1  : 0)
                    + (msb1_all1 & bit0 ? 1 : 0);

endmodule

```

Once the Verilog above was synthesised, it was imported into Qsys as a Nios II custom instruction component (the `ncs.*` port prefix lets Platform Designer auto-detect it as a custom instruction slave rather than a generic Avalon-MM slave) and connected to the Nios II core's `custom_instruction_master` interface, as shown at the bottom of Figure 2.1 (`count_leading_ones_0`, opcode 0). This exposes the circuit in software as the GCC built-in `__builtin_custom_ini(...)` custom instruction call, which the Nios II compiles down to a single `custom` instruction.

3.3.1 Hardware Performance Result

Running the equivalent test with the leading-ones count performed by the custom instruction instead of the C loop, for the same 100,000 iterations, gives an average execution time of 3.03 μ s per call:

```

Running performance test (100000 iterations)...
Performance test complete.
Total time    : 303000 us (303 ticks)
Avg per call  : 3.03 us
Last result   : 5

```

Unlike the software loop, this cost is constant regardless of the input word, since the combinational priority-encoder tree resolves every bit position in the same clock cycle no matter how many leading ones are present.

3.4 Comparison

The custom instruction is roughly $24.63/3.03 \approx 8.1\times$ faster on average than the software routine. This is the expected outcome: the software version executes a data-dependent loop of up to 32 iterations, each costing at least a shift, mask, compare, branch and increment, whereas the hardware version resolves the whole

Table 3.2: Software vs. hardware leading-ones count, 100,000 iterations

	Software (C loop)	Hardware (custom instruction)
Total time	2463000 μ s	303000 μ s
Avg per call	24.63 μ s	3.03 μ s

32-bit priority-encoder tree combinationally in a single cycle, with the remaining measured time in both cases dominated by the fixed overhead of the function/instruction call, loop bookkeeping in the benchmark harness, and `clock()` itself.

Software routine – pros: no additional FPGA logic resources are consumed, it is portable to any Nios II configuration (or indeed any target) without hardware changes, and it is trivial to modify or debug. Cons: execution time is both slower and data-dependent (worst-case 32 iterations for an all-ones word), and it consumes instruction-fetch and data-path bandwidth on the processor for every iteration of the loop.

Hardware custom instruction – pros: constant, single-cycle latency regardless of input value, and it frees the processor from executing a software loop at all. Cons: it consumes FPGA logic resources (comparators and multiplexers for the priority-encoder tree), occupies one of a limited number of custom instruction opcode slots, requires the Qsys system to be regenerated and the FPGA reprogrammed whenever it changes, and is tied to this specific Nios II system rather than being portable in software alone.

Overall, for a leading-ones count that is executed frequently or on a performance-critical path, the custom instruction is clearly the better choice, trading a small, fixed amount of FPGA fabric for an $\sim 8\times$ speed-up and deterministic timing. For infrequent or non-critical use, the software routine is preferable, since it avoids consuming custom instruction and logic resources that could otherwise be used elsewhere in the design.

Chapter 4

Part C - PWM Module

Chapter 5

Conclusions

Overall this assignment went well, I ran into some issues

Bibliography

- [1] Intel Corporation. Intel quartus prime pro edition user guide: Platform designer. <https://cdrdv2-public.intel.com/704996/ug-qpp-platform-designer-19-1-683609-704996.pdf>, 2026. Accessed: 2026-08-04. Platform Designer was formerly named Qsys.
- [2] Wikipedia contributors. Altera. <https://en.wikipedia.org/wiki/Altera>, 2026. Accessed: 2026-08-04.
- [3] Wikipedia contributors. Nios ii. https://en.wikipedia.org/wiki/Nios_II, 2026. Accessed: 2026-07-31.
- [4] Wikipedia contributors. Soft microprocessor. https://en.wikipedia.org/wiki/Soft_microprocessor, 2026. Accessed: 2026-07-31.